SMART EVENT MANAGEMENT & TICKETING PLATFORM

1. Case Study Introduction

The events and entertainment industry is undergoing rapid digital transformation, with organisations increasingly relying on online platforms to manage events, ticket sales, attendance and customer engagement. Many organisations still experience challenges such as overbooking, lack of system security, fragmented booking processes and limited access to real-time business insights.

You are part of a software development consultancy team contracted by Advanced Events (Pty) Ltd, a growing event management company that hosts corporate conferences, workshops, music festivals, and private events. The company currently relies on manual spreadsheets and disconnected online tools, which has resulted in booking errors, poor access control and operational inefficiencies.

Your task is to design and develop a Smart Event Management and Ticketing Platform that addresses these challenges. The solution must be secure, scalable and professionally designed to simulate a real-world production system.

2. Project Overview

This project requires you to design and implement a full-stack web application using Node.js, Express, EJS, and MongoDB. The system must simulate a real-world event booking platform with secure authentication, event management and ticket booking functionality.

3. Group Structure and Expectations

You must work in groups of five (5). Each group represents a professional software development team. All members are expected to contribute meaningfully to both frontend and backend development.

Recommended roles include:

- Team Lead / Project Coordinator
- Backend Developer
- Frontend Developer
- Database Engineer
- Security / DevOps Engineer

4. Functional Requirements

1. User Authentication: Registration, login, hashed passwords, role-based access
2. Event Management: Create, read, update, and delete events (admin only)
3. Ticket Booking System: Automated capacity control and booking validation
4. Dashboards: Admin analytics and user booking history
5. Contact Management System: Store and retrieve user enquiries
6. earch and Filtering: Event discovery features

5. Technical Requirements

The system must follow the MVC architecture with a clear separation of concerns. Authentication and authorization must be implemented using middleware. MongoDB with Mongoose must be used for data persistence. GitHub must be used for version control with meaningful commits and collaboration evidence.

6. Portal Structure (Mandatory Web Pages)

The Smart Event Management and Ticketing Platform **must consist of five (5) clearly structured and navigable web pages**, implemented using EJS templates and following the MVC architectural pattern. Each page must support the relevant functional requirements of the system.

1. **Home / Event Listing Page**
   - Displays all available events
   - Implements search and filtering (by date, category, availability)
   - Allows users to view event details and available tickets
2. **User Authentication Page**
   - Handles user registration and login
   - Implements hashed passwords and secure authentication
   - Supports role-based access control (Admin vs Standard User)
3. **Event Management Page (Admin Only)**
   - Allows administrators to create, update, view, and delete events (CRUD)
   - Manages event capacity and availability
   - Enforces authorization via middleware
4. **Booking & Dashboard Page**

- Enables users to book tickets with automated capacity validation
- Displays user booking history
- Provides admin analytics dashboards (total bookings, popular events, capacity usage)

5. **Contact / Enquiry Management Page**

   - Allows users to submit enquiries via a contact form
   - Stores enquiries in the database
   - Enables administrators to view and manage user enquiries

All five pages must be:

- Secure and role-aware
- Integrated with MongoDB via Mongoose
- Styled for usability and basic responsiveness
- Demonstrated during final presentation and evaluation

7. Marking Rubric (100 Marks)

Please take note that the marks you will get for each assessment are determined by the way you handle the questions asked during presentation and demonstration of your project.

Assessment Area	Criteria	Marks
Functionality	Authentication, event management, booking system, dashboards, contact system	25
Architecture & Code Quality	MVC structure, modularity, maintainability	20
Database Design	Schema design, relationships, validation	15
Authentication & Security	Password hashing, role-based access control	10
UI / UX	Usability, layout, responsiveness	10
GitHub Collaboration	Commit frequency, balanced contribution	5
Documentation	README quality and system setup clarity	10
Overall group cohesion during presentation	Organised, on time and professionalism	5

8. Recommended Technology Stack

Mandatory Technologies (Must Be Used)

You must use the following core technologies to ensure consistency, fairness, and assessability across all groups:

Backend

- Node.js – Server-side JavaScript runtime
- Express.js – Web application framework for routing and middleware

Frontend (Server-Side Rendering)

- EJS (Embedded JavaScript Templates)
- HTML5, CSS3

Database

- MongoDB (Local installation or MongoDB Atlas)
- Mongoose ODM

Mandatory Architectural & Development Practices

All groups must implement:

- MVC Architecture (Models, Views, Controllers)
- Middleware for:
  - Authentication
  - Authorization
  - Error handling
- Role-Based Access Control (e.g., Admin vs User)
- Password Hashing (e.g., bcrypt)
- Git & GitHub for version control and collaboration

These practices reflect industry standards and directly support assessment of architecture, security, and collaboration.

Recommended Supporting Libraries (Allowed & Encouraged)

You may use (but are not limited to):

- bcrypt – Password hashing
- express-session or jsonwebtoken (JWT) – Authentication handling
- dotenv – Environment variable management
- nodemon – Development convenience
- bootstrap or tailwindcss – UI styling

These libraries improve security and usability without changing core system behaviour.

Tools & Environment

Students are encouraged to use:

- Visual Studio Code (or equivalent IDE)
- Postman / Thunder Client/ REST Client for API testing
- MongoDB Compass for database inspection

Technologies NOT Allowed

To maintain assessment consistency:

- React, Angular, or Vue
- Firebase as the primary backend
- PHP, Laravel, or Django
- Low-code / no-code platforms

9. Submission Instructions

Submit a ZIP file **excluding the node_modules folder**. The submission must include the complete source code, a comprehensive README file, and a GitHub repository link. The application must run without errors.

Your Read me file must include:

- Project Title and Overview
- Technologies Used
- Team Members and Roles
- 🛠 Setup Instructions (npm install, npm run dev)
- 🖼 Screenshots (optional)
- Reflection (optional)

10. Academic Integrity Clause

All students are required to adhere to the principles of **academic integrity** throughout the duration of this project. Academic integrity is fundamental to learning and includes honesty, trust, fairness, respect, and responsibility.

- The submitted project must be the original work of the group members listed and must not include plagiarised content from online sources, previous student submissions, or third-party repositories.
- While students may consult external resources (such as documentation, tutorials, and reference materials) to support learning, these resources must not be copied verbatim and must be properly adapted and understood by the group.
- The use of AI-assisted tools (e.g. for code suggestions or explanations) is permitted for learning purposes; however, students must be able to clearly explain and justify all code and design decisions during the presentation and questioning phase.
- Each group member is expected to contribute meaningfully to the project. Unequal participation or misrepresentation of individual contributions constitutes academic misconduct.
- Any form of code sharing between groups, falsification of contribution evidence, or misrepresentation during demonstrations will be treated as a serious breach of academic integrity.
- All submissions will be made via Moodle, and no late submissions will be accepted. When one member of the group submits it automatically submits for all the group members.

Failure to comply with this clause may result in:

- Mark penalties,
- A mark of zero for the assessment, and/or
- Disciplinary action in accordance with institutional policies.